UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Tonalá (CUTonalá)
División de Ingenierías e Innovación Tecnológica
Licenciatura en Ingeniería en Computación

Actividad 3.4. Buenas prácticas de calidad y sus estándares

Materia: Proyecto Integrador de Desarrollo de Software

Estudiante: Ernesto Hatuey Fierro Medina (Código: 215487963)

Docente: Dr. Gabriel Navarro Salcedo

Fecha de entrega: 13 de octubre de 2026

1. Introducción

En la formación como ingenieros en computación es común pensar que la calidad de un software se reduce a que el código compile, ejecute sus funciones principales y no presente caídas visibles durante una demostración rápida. Sin embargo, conforme se avanza en proyectos de desarrollo reales se hace evidente que un sistema puede funcionar hoy y resultar inmantenible o vulnerable mañana si no se construye sobre bases técnicas sólidas. La calidad en el software no es un evento aislado que se verifique al final del semestre; es una disciplina continua que debe integrarse en cada etapa del ciclo de vida del producto.

Para lograr que una aplicación sea robusta, escalable y mantenible en el tiempo, la ingeniería de software moderna articula tres niveles de calidad complementarios. En primer lugar, a nivel de diseño y código fuente, los principios SOLID formulados por Robert C. Martin (2003, 2018) orientan la estructura de clases y módulos para evitar la rigidez y el acoplamiento perjudicial. En segundo lugar, a nivel de evaluación del producto terminado, la norma internacional ISO/IEC 25010:2011 (ISO/IEC, 2011) proporciona un marco sistemático para definir y medir las características de calidad tanto internas como externas (seguridad, fiabilidad, usabilidad y eficiencia). En tercer lugar, a nivel de ciclo de vida operativo y entrega continua, las prácticas y la cultura DevOps (Bass et al., 2015; Kim et al., 2016) permiten automatizar pruebas, empaquetar aplicaciones y desplegar versiones de forma repetible mediante pipelines de integración continua (Humble & Farley, 2010).

En este trabajo analizo de forma técnica estos tres pilares de calidad, presento una tabla comparativa con sus conceptos clave y desarrollo una segunda tabla detallando cómo aplico concretamente cada estándar y buena práctica en el desarrollo individual de mi proyecto integrador, la plataforma web Monchis Café.

2. Resumen técnico de buenas prácticas y estándares

2.1. Principios de diseño orientado a objetos: SOLID

El acrónimo SOLID reúne cinco principios fundamentales de diseño orientado a objetos recopilados por Robert C. Martin (2003, 2018). Su propósito es guiar la creación de software modular, comprensible y preparado para evolucionar sin generar efectos secundarios no deseados ante futuros requerimientos:

• **Single Responsibility Principle (SRP - Responsabilidad Única):** Establece que una clase o módulo debe tener una sola razón para cambiar, lo que significa que debe encapsular una única responsabilidad de negocio. Si una clase maneja simultáneamente reglas de cálculo, formato visual de presentación y persistencia en base de datos, cualquier cambio en una de esas áreas puede alterar accidentalmente las otras.

• **Open/Closed Principle (OCP - Abierto/Cerrado):** Formulado originalmente por Bertrand Meyer y adaptado por Martin (2018), indica que las entidades de software deben estar abiertas a la extensión, pero cerradas a la modificación. Esto se logra mediante abstracciones e interfaces: para agregar un nuevo comportamiento o método de pago, se crea una nueva clase que implemente la interfaz existente, en lugar de modificar código ya probado y en producción.

• **Liskov Substitution Principle (LSP - Sustitución de Liskov):** Propuesto por Barbara Liskov (Liskov & Wing, 1994), señala que los objetos de una subclase deben poder sustituir a los objetos de su clase base sin alterar la correctitud del programa. Las clases derivadas deben respetar los contratos, precondiciones y postcondiciones fijados por la abstracción original.

• **Interface Segregation Principle (ISP - Segregación de Interfaces):** Dictamina que los clientes no deben ser forzados a depender de interfaces que contienen métodos que no utilizan. Es preferible diseñar múltiples interfaces pequeñas y específicas a tener una interfaz general y pesada, evitando que los módulos implementen código vacío o innecesario.

• **Dependency Inversion Principle (DIP - Inversión de Dependencias):** Establece que los módulos de alto nivel (la lógica central de negocio) no deben depender de módulos de bajo nivel (bases de datos, controladores de red o librerías externas); ambos deben depender de abstracciones. Asimismo, las abstracciones no deben depender de los detalles técnicos, sino los detalles de las abstracciones.

2.2. Modelo de calidad del producto software: ISO/IEC 25010

La norma internacional ISO/IEC 25010:2011 (ISO/IEC, 2011), integrada dentro de la familia de estándares SQuaRE (Software Product Quality Requirements and Evaluation), define un modelo integral para descomponer y evaluar la calidad del software en ocho características fundamentales, cada una compuesta por subcaracterísticas verificables:

1. **Adecuación funcional:** Grado en que el software proporciona funciones que satisfacen las necesidades declaradas bajo condiciones específicas (completitud, corrección y pertinencia funcional).
2. **Eficiencia de desempeño:** Rendimiento del software relativo a los recursos utilizados bajo condiciones establecidas (comportamiento temporal o latencia, utilización de CPU/memoria y capacidad máxima de procesamiento).
3. **Compatibilidad:** Capacidad de dos o más sistemas o componentes para intercambiar información y ejecutar funciones compartiendo el mismo entorno de hardware o software (coexistencia e interoperabilidad mediante APIs).
4. **Usabilidad:** Facilidad con la que usuarios específicos pueden utilizar el software para lograr objetivos con efectividad, eficiencia y satisfacción (reconocibilidad de funciones, capacidad de aprendizaje, operabilidad, protección contra errores de usuario, estética de la interfaz y accesibilidad).
5. **Fiabilidad:** Capacidad del sistema para mantener un nivel especificado de desempeño bajo condiciones fijas durante un tiempo determinado (madurez, disponibilidad de servicio, tolerancia a

fallos y capacidad de recuperación o resiliencia).
6. **Seguridad:** Grado en que el sistema protege la información y los datos para que personas o sistemas no autorizados no puedan acceder ni modificarlos (confidencialidad, integridad de datos, no repudio, autenticidad y responsabilidad o auditoría).
7. **Mantenibilidad:** Efectividad y facilidad con la que el software puede ser modificado por los desarrolladores para corregirlo, mejorarlo o adaptarlo a cambios del entorno (modularidad, reusabilidad, analizabilidad, modificabilidad y testabilidad o facilidad de prueba).
8. **Portabilidad:** Facilidad con la que el software puede ser transferido de un entorno de hardware, software u operativo a otro (adaptabilidad, instalabilidad y capacidad de reemplazo).

2.3. Paradigma y prácticas DevOps

DevOps es una combinación de filosofías culturales, prácticas de ingeniería y herramientas de automatización que integra el desarrollo de software (Dev) con las operaciones de tecnología (Ops) para acortar el ciclo de vida del desarrollo y entregar valor continuo con alta calidad (Bass et al., 2015; Kim et al., 2016). Entre sus prácticas más representativas destacan:

• **Integración Continua (CI):** Los desarrolladores fusionan sus cambios de código con frecuencia en un repositorio central. Cada integración desencadena automáticamente la compilación del proyecto, la ejecución de analizadores estáticos de código (linters) y la ejecución completa de suites de pruebas automatizadas (pruebas unitarias y de integración) para detectar errores en etapas tempranas (Humble & Farley, 2010).
• **Entrega y Despliegue Continuo (CD):** Extensión de CI donde cada cambio aprobado en el código se empaqueta de forma automática (por ejemplo, en imágenes de contenedores) y se despliega sin intervención manual hacia entornos de prueba (staging) o producción, reduciendo el riesgo de errores humanos durante la puesta en marcha.
• **Infraestructura como Código (IaC):** Gestión y aprovisionamiento de servidores, redes, bases de datos y balanceadores mediante archivos de configuración declarativos auditables en control de versiones (por ejemplo, con Docker Compose o Terraform), garantizando que los entornos de desarrollo, pruebas y producción sean idénticos y reproducibles.
• **Observabilidad, Monitoreo y Logging Centralizado:** Recopilación estructurada de registros (logs), métricas de rendimiento y trazas de ejecución distribuidas para supervisar la salud del sistema en tiempo real y diagnosticar incidencias rápidamente.

3. Tabla comparativa de buenas prácticas y estándares

La siguiente tabla sintetiza la definición, el propósito, las características clave y el impacto en el ciclo de vida del software de los tres enfoques de calidad analizados:

Estándar o práctica	Nivel de aplicación	Definición y concepto clave	Propósito principal	Pilares o características centrales	Impacto en el ciclo de vida del software
Principios SOLID (Robert C. Martin)	Nivel de código fuente y diseño orientado a objetos.	Cinco directrices de diseño estructurado que guían la relación entre clases, métodos y módulos para evitar dependencias frágiles (Martin, 2018).	Reducir el acoplamiento y aumentar la cohesión, permitiendo que el software crezca sin que un cambio local rompa otras partes del sistema.	• Single Responsibility (SRP) • Open/Closed (OCP) • Liskov Substitution (LSP) • Interface Segregation (ISP) • Dependency Inversion (DIP)	Facilita la fase de desarrollo y mantenimiento continuo; permite crear pruebas unitarias aisladas mediante mocks y reduce drásticamente el costo de refactorización.
Norma ISO/IEC 25010 (SQuaRE - ISO/IEC, 2011)	Nivel de producto software terminado y especificación de calidad.	Estándar internacional que descompone y normaliza la calidad del software en un conjunto estructurado de características cuantificables.	Definir requisitos no funcionales objetivos, orientar las pruebas de aceptación y auditar la calidad integral percibida por los usuarios y operadores.	8 características de calidad: 1. Adecuación funcional 2. Eficiencia de desempeño 3. Compatibilidad 4. Usabilidad 5. Fiabilidad 6. Seguridad 7. Mantenibilidad 8. Portabilidad	Guía la fase de análisis de requisitos no funcionales, define los criterios de aceptación durante las pruebas de integración y valida el cumplimiento formal del producto.
Metodología DevOps (Bass et al., 2015; Kim et al., 2016)	Nivel de proceso, ciclo de vida operativo, despliegue e infraestructura.	Cultura y conjunto de prácticas automatizadas que integran el desarrollo de software con las operaciones de infraestructura y entrega (Humble & Farley, 2010).	Acelerar el tiempo de entrega de software al mercado (time-to-market), garantizando despliegues estables, predecibles y de bajo riesgo.	• Integración Continua (CI) • Entrega Continua (CD) • Infraestructura como Código (IaC) • Contenedorización • Observabilidad y logging centralizado	Abarca desde la fase de construcción hasta la operación en producción; automatiza las pruebas ante cada commit, elimina errores manuales de despliegue y asegura monitoreo continuo.

4. Aplicación práctica al proyecto integrador: Monchis Café

En la siguiente tabla se detalla la aplicación técnica de los principios SOLID, la norma ISO/IEC 25010 y las prácticas DevOps dentro de los módulos y componentes reales de mi proyecto integrador Monchis Café:

Estándar o práctica	Componente o módulo de Monchis Café	Problema o riesgo atendido	Implementación técnica en el proyecto	Métrica o criterio de verificación
SOLID (Single Responsibility)	Backend: Handlers de API y servicios de venta (Next.js y Prisma).	Acoplamiento indebido si un controlador de ruta maneja validación HTTP, cálculo de totales, lógica de cobro y queries a base de datos.	Separación en tres capas independientes: 1) Handlers HTTP para parsear requests; 2) Servicios de dominio para reglas comerciales de caja; 3) Repositorios de datos para queries con Prisma ORM.	Cada archivo de servicio tiene menos de 200 líneas de código y una sola razón de cambio; cambios de base de datos no alteran los endpoints HTTP.
SOLID (Open/Closed)	Punto de Venta: Módulo de procesamiento de pagos (POS).	Riesgo de modificar código productivo cada vez que se agrega un método de pago (efectivo, tarjeta, SPEI o Monchis Rewards).	Definición de una interfaz común `IPaymentProcessor`. Cada forma de pago se implementa como una clase concreta independiente sin modificar el orquestador principal de checkout.	Incorporación de un nuevo método de pago (ej. monedero digital) creando únicamente una clase nueva y pruebas aisladas, con cero modificaciones al checkout.
SOLID (Liskov Substitution)	Eventos asíncronos y Patrón Saga (RabbitMQ).	Fallas en el consumidor de eventos si un evento derivado de compensación/reversa rompe el contrato del evento base de venta.	Todos los eventos del dominio heredan de la clase abstracta `DomainEvent`. Cualquier evento de reversa puede ser procesado por el despachador de eventos sin alterar su comportamiento.	100 % de los eventos de compensación pasan las pruebas de serialización y consumo en Vitest sin lanzar excepciones inesperadas de tipo.
SOLID (Interface Segregation)	Frontend: Vistas y componentes en Vue 3 (Pinia Stores).	Componentes sobrecargados que reciben interfaces monolíticas con métodos y propiedades que no utilizan.	Segregación de interfaces en TypeScript: `ICatalogoPublico` para clientes, `IPuntoVentaCajero` para barra de cobro y `IAdminDashboard` para métricas de gerencia.	Los componentes Vue 3 del catálogo público no tienen dependencias hacia funciones de caja ni de apertura de turno, reduciendo el bundle de JavaScript.
SOLID (Dependency Inversion)	Servicios de mensajería y persistencia.	Acoplamiento directo a la librería `amqplib` de RabbitMQ o a Prisma, impidiendo realizar pruebas unitarias rápidas sin conexión real.	Los servicios de negocio dependen de la interfaz abstracta `IMessageBroker`. La conexión real con RabbitMQ se inyecta en tiempo de ejecución.	Capacidad de ejecutar 100 % de las pruebas unitarias de negocio en memoria en menos de 3 segundos utilizando mocks del broker de mensajería.
ISO/IEC 25010 (Seguridad)	Módulo de Autenticación Stateless y Control de Acceso (RBAC).	Robo de credenciales, suplantación de identidad en barra de cobro y ataques automatizados de bots.	Tokens JWT rotativos con refresh token en cookies seguras (`httpOnly, Secure, SameSite=Strict`), verificación Google reCAPTCHA v3 y roles diferenciados (admin, cajero, cliente).	Reportes de análisis estático SAST libres de vulnerabilidades críticas y bloqueo automático del 100 % de accesos no autorizados a endpoints protegidos.

Estándar o práctica	Componente o módulo de Monchis Café	Problema o riesgo atendido	Implementación técnica en el proyecto	Métrica o criterio de verificación
ISO/IEC 25010 (Fiabilidad)	Broker RabbitMQ y Patrón Saga (Transacciones compensatorias).	Inconsistencia transaccional si se cobra una venta en caja pero el lote de café orgánico regional se agotó físicamente en almacén.	Orquestación Saga asíncrona: ante falla de inventario, se emite automáticamente una transacción compensatoria de reembolso y desvío a colas Dead Letter Queue (DLQ).	Recuperación garantizada de consistencia eventual en menos de 2 segundos ante fallos simulados en pruebas de integración, con cero ventas huérfanas.
ISO/IEC 25010 (Usabilidad)	Interfaz táctil del Punto de Venta (POS) en Vue 3.	Errores de tecleo de los cajeros en horas pico y lentitud en el cobro de bebidas y alimentos.	Diseño cálido pastel con botones grandes de alto contraste táctil, lectura rápida de código de barras/QR y cálculo automático de cambio en efectivo.	Tiempo de registro de una venta de café en mostrador completado en menos de 15 segundos en las pruebas de validación con el encargado de cafetería.
ISO/IEC 25010 (Eficiencia)	Catálogo de Café Orgánico y Trazabilidad (Vite-SSG y PostgreSQL).	Lentitud en la carga web desde dispositivos móviles y cuellos de botella al consultar historial de cosechas y lotes.	Prerenderizado estático con `vite-ssg` para páginas de catálogo e indexación de base de datos en campos críticos (`fecha_caducidad`, `finca_origen`, `numero_lote`).	Puntaje de rendimiento en Google Lighthouse superior a 90 puntos y tiempos de respuesta de consulta de trazabilidad inferiores a 100 milisegundos.
ISO/IEC 25010 (Mantenibilidad)	Arquitectura Monorepo (Turborepo) y Pruebas con TDD.	Degradación del código con el tiempo y temor a introducir regresiones al actualizar dependencias compartidas.	Estructura monorepo con paquetes tipados compartidos en TypeScript, linter estricto (ESLint/Prettier) y desarrollo guiado por pruebas (TDD) en Vitest.	Cobertura de pruebas automatizadas superior al 85 % en la lógica transaccional y pipelines de compilación limpios sin advertencias de tipado.
DevOps (Integración Continua - CI)	Pipeline de GitHub Actions del repositorio.	Integración de código defectuoso que rompa la rama principal o introduzca fallas inadvertidas al repositorio.	Pipeline automatizado que se ejecuta en cada push o pull request: valida sintaxis con `pnpm lint`, verifica tipos con `tsc` y corre suites completas de pruebas unitarias.	Bloqueo automático de commits si alguna prueba unitaria falla; tiempo total de ejecución del pipeline de integración inferior a 3 minutos.
DevOps (Contenedorización y CD)	Empaquetado de aplicaciones (Frontend Vue 3 y Backend Next.js).	El problema clásico de "en mi máquina sí funciona" debido a diferencias en versiones de Node.js o librerías del sistema operativo.	Dockerfiles multi-stage para generar imágenes ligeras de producción que aíslan las dependencias de ejecución de las herramientas de compilación.	Imágenes Docker de producción con tamaño reducido (menos de 180 MB para el backend) y arranque de contenedores garantizado en cualquier servidor.
DevOps (Infraestructura como Código)	Entorno local de desarrollo y servicios de soporte.	Configuración manual tediosa y propensa a errores de bases de datos PostgreSQL y colas RabbitMQ en máquinas nuevas.	Archivo `docker-compose.yml` que orquesta PostgreSQL con extensiones, broker RabbitMQ con management plugin y variables de entorno parametrizadas.	Levantamiento completo del entorno de desarrollo funcional y listo para pruebas con un único comando (`docker compose up -d`).
DevOps (Observabilidad y Logging)	Trazabilidad de eventos distribuidos de venta y reversa.	Dificultad para diagnosticar qué paso de una transacción Saga	Inyección de un identificador único de correlación (`correlation_id`) en cada mensaje de RabbitMQ y registro estructurado en logs de consola.	100 % de los eventos de venta y reversa rastreables de inicio a fin en los logs mediante su

Estándar o práctica	Componente o módulo de Monchis Café	Problema o riesgo atendido	Implementación técnica en el proyecto	Métrica o criterio de verificación
		falló cuando intervienen múltiples colas de mensajería asíncrona.		identificador único de transacción.

5. Conclusiones generales

El análisis desarrollado en esta actividad me permitió comprender que la calidad en la ingeniería de software no es un concepto teórico aislado, sino un engranaje donde interactúan tres niveles indispensables. Los principios SOLID operan en el nivel más cercano al programador: garantizan que la arquitectura interna del código sea limpia, desacoplada y fácil de probar. Por su parte, la norma ISO/IEC 25010 aporta la perspectiva del producto terminado: fija los criterios objetivos para evaluar si el software es seguro, confiable, eficiente y agradable de utilizar para las personas. Finalmente, la cultura y herramientas DevOps cierran la brecha operativa: automatizan las verificaciones de calidad y aseguran que el código probado pase a producción sin depender de procesos manuales propensos a fallos.

Al desarrollar mi proyecto integrador Monchis Café de forma individual, adoptar este enfoque integral de calidad no representa un lujo ni una carga burocrática adicional, sino la única estrategia viable para mantener bajo control un sistema de complejidad considerable. Integrar colas de mensajería asíncrona con RabbitMQ y el patrón Saga para la reversa de cobros, junto con un punto de venta táctil y trazabilidad de café orgánico, sería sumamente riesgoso si el código estuviera fuertemente acoplado o si tuviera que probar manualmente cada pantalla antes de una entrega.

La aplicación práctica demostrada en la Tabla 2 evidencia beneficios inmediatos: la inversión de dependencias y la responsabilidad única de SOLID me permiten escribir pruebas unitarias rápidas con Vitest sin necesidad de tener encendido el broker real de mensajería; las características de seguridad y fiabilidad de la ISO 25010 protegen las ventas concurrentes y las credenciales de caja; y los pipelines de integración continua de DevOps me dan la certeza de que ningún commit introducirá regresiones inadvertidas al monorepo. En conclusión, estas buenas prácticas y estándares constituyen el cimiento técnico que me permite construir una plataforma profesional, mantenible y verdaderamente útil para las operaciones comerciales de Monchis Café.

Referencias

Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A software architect's perspective*. Addison-Wesley.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

International Organization for Standardization, & International Electrotechnical Commission. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC Standard No. 25010:2011). ISO/IEC.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.

Liskov, B., & Wing, J. M. (1994). A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems, 16*(6), 1811–1841. https://doi.org/10.1145/197320.197383

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.